

Livrable 3 : Replica Set

Far Adam INFO FISE 4A

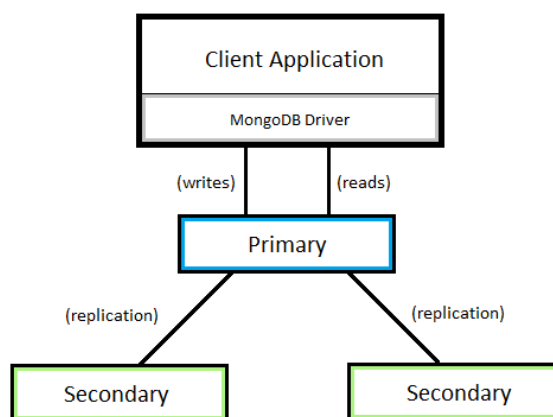
Introduction

Ce troisième livrable s'inscrit dans le cadre du projet CinéExplorer, dont l'objectif est de concevoir une plateforme hybride de découverte de films utilisant à la fois SQLite et MongoDB. Après avoir exploré les données IMDB et mis en place une base de données NoSQL performante lors des phases précédentes, cette étape se concentre sur un pilier fondamental des architectures modernes qui est la haute disponibilité. Dans un environnement de production, une base de données ne peut se permettre d'être un point unique de défaillance. L'objectif de cette Phase 3 est donc de transformer notre instance MongoDB isolée en un Replica Set distribué. Cette architecture, composée de trois nœuds distincts, permet d'assurer une continuité de service et une intégrité des données même en cas de panne matérielle ou logicielle.

Architecture du cluster

L'architecture de ce cluster repose sur la présence de bases de données représentées par des nœuds. Ces nœuds se voient chacun attribués un rôle qui est primaire ou secondaire. Le nœud primaire est le seul nœud qui accepte les opérations d'écritures. Les nœuds secondaires répliquent le contenu des écritures.

On peut représenter l'architecture par le schéma ci-contre :



Configuration des nœuds :

Pour configurer les nœuds, on lance le script setup_replica :

Le script ouvre 3 terminaux distinct :

Dans chaque terminal, on crée un nœud via le programme principal de mongo. On le définit comme appartenant à un replica set nommé rs0. On lui attribue un numéro de port. On définit le chemin où les données vont être enregistrées et on garantit que les connexions ne viendront pas d'internet.

On utilise dans l'ordre les commandes suivantes :

```
mongod --replSet rs0 --port 27017 --dbpath ./data/mongo/db-1 --bind_ip localhost
```

```
mongod --replSet rs0 --port 27018 --dbpath ./data/mongo/db-2 --bind_ip localhost
```

```
mongod --replSet rs0 --port 27019 --dbpath ./data/mongo/db-3 --bind_ip localhost
```

Puis on remplit la base de données primaire en lançant les scripts python de SQLite et de migration de replica set MongoDB.

Puis pour se connecter à la base du port 27017 :

```
mongosh --port 27017
```

Tests et tolérance aux pannes :

1. Etat Initial

```
rs.status()
```

On demande le statut des nœuds :

Et on voit que le port 27017 est primaire et les port 27018 et 27019 sont secondaires :

```
members: [
  {
    _id: 0,
    name: 'localhost:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 136,
    optime: { ts: Timestamp({ t: 1767355468, i: 1 }), t: Long('26') },
    optimeDurable: { ts: Timestamp({ t: 1767355468, i: 1 }), t: Long('26') },
    optimeDate: ISODate('2026-01-02T12:04:28.000Z'),
    optimeDurableDate: ISODate('2026-01-02T12:04:28.000Z'),
    lastAppliedWallTime: ISODate('2026-01-02T12:04:28.501Z'),
    lastDurableWallTime: ISODate('2026-01-02T12:04:28.501Z'),
    lastHeartbeat: ISODate('2026-01-02T12:04:29.534Z'),
    lastHeartbeatRecv: ISODate('2026-01-02T12:04:30.532Z'),
    pingMs: Long(0),
    lastHeartbeatMessage: '',
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1767355337, i: 1 }),
    electionDate: ISODate('2026-01-02T12:02:17.000Z'),
    configVersion: 1,
    configTerm: 26
  },
]
```

```
{
  _id: 1,
  name: 'localhost:27018',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 141,
  optime: { ts: Timestamp({ t: 1767355468, i: 1 }), t: Long('26') },
  optimeDate: ISODate('2026-01-02T12:04:28.000Z'),
  lastAppliedWallTime: ISODate('2026-01-02T12:04:28.501Z'),
  lastDurableWallTime: ISODate('2026-01-02T12:04:28.501Z'),
  syncSourceHost: 'localhost:27017',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 1,
  configTerm: 26,
  self: true,
  lastHeartbeatMessage: ''
},
```

```
{
  _id: 2,
  name: 'localhost:27019',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 124,
  optime: { ts: Timestamp({ t: 1767355468, i: 1 }), t: Long('26') },
  optimeDurable: { ts: Timestamp({ t: 1767355468, i: 1 }), t: Long('26') },
  optimeDate: ISODate('2026-01-02T12:04:28.000Z'),
  optimeDurableDate: ISODate('2026-01-02T12:04:28.000Z'),
  lastAppliedWallTime: ISODate('2026-01-02T12:04:28.501Z'),
  lastDurableWallTime: ISODate('2026-01-02T12:04:28.501Z'),
  lastHeartbeat: ISODate('2026-01-02T12:04:30.871Z'),
  lastHeartbeatRecv: ISODate('2026-01-02T12:04:29.683Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: 'localhost:27018',
  syncSourceId: 1,
  infoMessage: '',
  configVersion: 1,
  configTerm: 26
}
```

2.Ecriture

Ensuite, on écrit dans le nœud primaire grâce au script setup_replica.bat.

On vérifie que les données sont répliquées dans les nœuds secondaires :

```
rs0 [direct: secondary] imdb_flat> show collections
characters
directors
genres
known_for
movies
movies_complete
persons
principals
professions
ratings
titles
writers
```

3 et 4. Panne Primary et Panne Primary

On effectue le ctrl + C sur le primaire et on observe qu'un nouveau nœud est élu.

```
members: [
  {
    _id: 0,
    name: 'localhost:27018',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 1363,
    optime: { ts: Timestamp({ t: 1767356002, i: 1 }), t: Long('27') },
    optimeDurable: { ts: Timestamp({ t: 1767356002, i: 1 }), t: Long('27') },
    optimeDate: ISODate('2026-01-02T12:24:02.000Z'),
    optimeDurableDate: ISODate('2026-01-02T12:24:02.000Z'),
    lastAppliedWallTime: ISODate('2026-01-02T12:24:02.853Z'),
    lastDurableWallTime: ISODate('2026-01-02T12:24:02.853Z'),
    lastHeartbeat: ISODate('2026-01-02T12:24:09.081Z'),
    lastHeartbeatRecv: ISODate('2026-01-02T12:24:09.086Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: 'localhost:27019',
    syncSourceId: 2,
    infoMessage: '',
    configVersion: 1,
    configTerm: 27
  },
  {
    _id: 1,
    name: 'localhost:27019',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 1363,
    optime: { ts: Timestamp({ t: 1767356002, i: 1 }), t: Long('27') },
    optimeDate: ISODate('2026-01-02T12:24:02.000Z'),
    lastAppliedWallTime: ISODate('2026-01-02T12:24:02.853Z'),
    lastDurableWallTime: ISODate('2026-01-02T12:24:02.853Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1767356002, i: 1 }),
    electionDate: ISODate('2026-01-02T12:22:02.000Z'),
    configVersion: 1,
    configTerm: 27,
    self: true,
    lastHeartbeatMessage: ''
  }
]
```

Pour vérifier que l'infrastructure est tolérante aux pannes, on effectue sur ces nœuds un test de panne. Cela prouve que lorsque le nœud primaire s'arrête, un nœud secondaire prend le relais. On simule donc un incident sur le primaire. On utilise le script test_failover.py qui quand on tue le nœud primaire détecte l'élection du nouveau primaire et mesure le temps de latence pendant lequel le nœud ne répond pas.

```
✅ Élection terminée !
⌚ Temps de failover réel (arrêt du service d'écriture) : 13.82 secondes
```

Le temps renvoyé est proche de 14s. Or, le temps que met MongoDB pour considérer qu'un nœud est mort est environ 10s. l'élection prend donc 4s, ce qui semble réaliste.

5.6. Lecture et Reconnexion

Les données sont toujours accessibles lorsqu'on se connecte sur le nouveau primaire et lorsqu'on reconnecte l'ancien nœud primaire qui a été tué, on remarque qu'il devient un secondaire.

7. Double panne

En cas de double panne, on constate que le dernier nœud restant ne devient pas un nœud principal et reste un nœud secondaire. Cela s'explique par le fait que l'élection doit s'effectuer à la majorité stricte et que lorsqu'il n'y a plus qu'un seul nœud, cette majorité ne peut pas être atteinte.

Conclusion :

La réalisation de cette Phase 3 a permis de transformer notre base de données CinéExplorer en un système distribué et résilient. Grâce à la mise en place d'un Replica Set à trois nœuds, nous avons atteint les objectifs de haute disponibilité essentiels pour une application moderne.

Le test de failover a validé la robustesse de l'architecture : suite à la simulation d'une panne critique du nœud primaire, le système a su s'auto-organiser pour élire un nouveau leader et restaurer le service d'écriture sans intervention humaine. Cette expérience met en évidence le compromis fondamental opéré par MongoDB entre disponibilité et cohérence.